

Reviewer Pack — Real Radical Dimension Bounds SOS Degree for Max-CUT

Entry 4b5de421e063 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 22:53:13 UTC

Real Radical Dimension Bounds SOS Degree for Max-CUT

Entry ID: 4b5de421e063 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 22:53:13 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random 3-CNF instance φ on n variables, let I_φ be the ideal generated by its Max-CUT constraints and let $\dim(\sqrt{I}_\varphi)$ denote the dimension of its real radical. Then the SOS degree required to approximate φ 's Max-CUT value within $(1-\varepsilon)$ is $\Omega(\dim(\sqrt{I}_\varphi) / \log n)$ for $\varepsilon = 1/n^2$.

Rationale (proposer's reasoning):

The real radical captures the algebraic structure of feasible solutions; higher-dimensional radicals imply more complex solution manifolds requiring higher-degree SOS certificates. This links geometric complexity to proof system strength, potentially revealing new barriers for SOS approximations.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8e87ed485ec429e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(10 * n): # Each variable appears in about 10 clauses
            variables = [random.randint(0, n-1) for _ in range(3)]
            clause = [(-1 if random.choice([True, False]) else 1) * (x + 1) for x in variables]
            clauses.append(clause)
        return clauses

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a*b) // gcd(a, b)

    def matrix_multiply(A, B):
        m, n = len(A), len(B[0])
        p = len(B)
        C = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        m, n = len(A), len(A[0])
        augmented = [A[i] + [b[i]] for i in range(m)]
        for i in range(n):
            max_row = i
            for j in range(i+1, m):
                if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                    max_row = j
```

```

        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        pivot = augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
                    augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]

def solve_linear_system(A, b):
    return gaussian_elimination(A, b)

def is_integer(x):
    return abs(x - round(x)) < 1e-9

n = 40
clauses = generate_3cnf(n)

# Convert clauses to a system of polynomial equations
variables = [i for i in range(2*n)]
A = []
b = []
for clause in clauses:
    row = [0] * (2*n)
    for var in clause:
        if var > 0:
            row[var-1] += 1
        else:
            row[-var-1] -= 1
    A.append(row)
    b.append(0)

# Solve the system to find the dimension of the radical
solution = solve_linear_system(A, b)
rank = sum(is_integer(x) for x in solution)
dim_radical = n - rank

# Estimate the minimal SOS degree required
epsilon = 1 / (n**2)
d = dim_radical * math.log(n)

return {
    "metric_name": "SOS_degree",
    "metric_value": d,
    "instances_tested": 1,
    "conjecture_holds": d >= 0.5 * dim_radical / math.log(n),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7146607.py", line 112, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7146607.py", line 90, in run_trial
    solution = solve_linear_system(A, b)
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7146607.py", line 67, in solve_linear_
    return gaussian_elimination(A, b)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7146607.py", line 58, in gaussian_elim
    augmented[i][j] /= pivot
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing result evaluation | next: Fix division-by-zero in gaussian_elimination and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	63868
2	preregistration	ollama_remote	qwen3:8b	0	0	32570
3	novelty	ollama_remote	qwen3:8b	0	0	27970
4	novelty	ollama_remote	qwen3:8b	0	0	19295
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23922
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7981
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9814
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13147
9	judge	ollama_remote	qwen3:8b	0	0	22273

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 220840 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4b5de421e063.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b5de421e063.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4b5de421e063.tar.gz` (if generated)